

Module «Data Science»

Week 04: Part II



mario.gellrich@zhaw.ch

Dr. Mario Gellrich

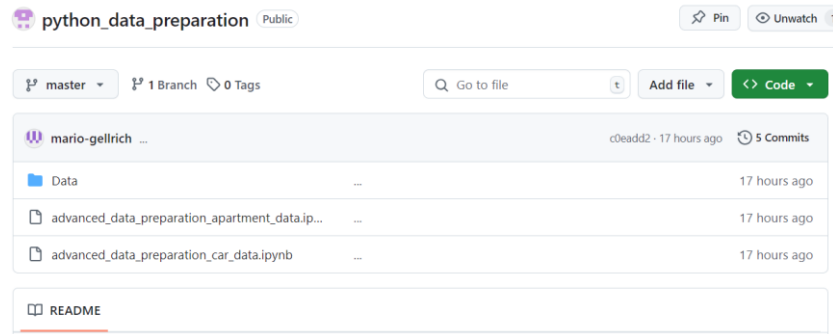
Content of this lecture

Table of contents:

- Advanced Data Preparation Methods
- Combining and Organizing Data

Software used in this Course

GitHub Repository with Course Material for Python exercises



[Open the python_data_preparation repository on GitHub](#)

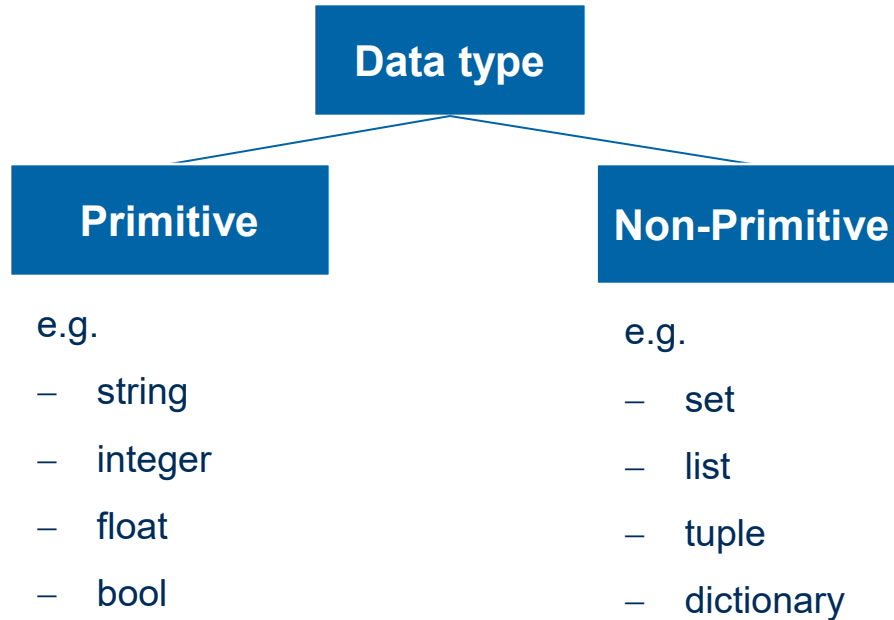
Tasks:

- Fork the repository into your own GitHub account
- Create a new Codespaces environment

Advanced Data Preparation Methods

- Recap: Data Types and Data Structures in Python
- Recap: Scale of Measure
- Data Cleansing
- String Manipulation
- Date Formatting
- Data Discretization
- One-Hot Encoding
- Data Scaling
- Data Standardization
- Data Transformation (Statistics)

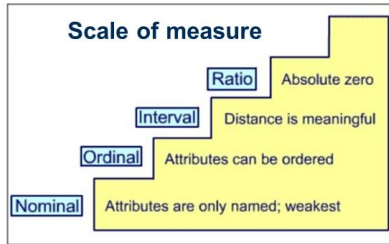
Recap: Data Types and Data Structures in Python



A **data type** is an attribute of data which tells the compiler or interpreter how the programmer intends to use the data. Data types can be divided into **primitive** and **non-primitive** data types. Non-primitive data types are also referred to as composite data types or data structures.

Recap: Scale of Measure – choice of statistical method

The choice of the correct statistical method is made on the basis of data types



Examples:

10 kg

36 degree Celsius

bad, neutral, good

cat, dog, bird

Numerical data

Ordinal data

Categorical data

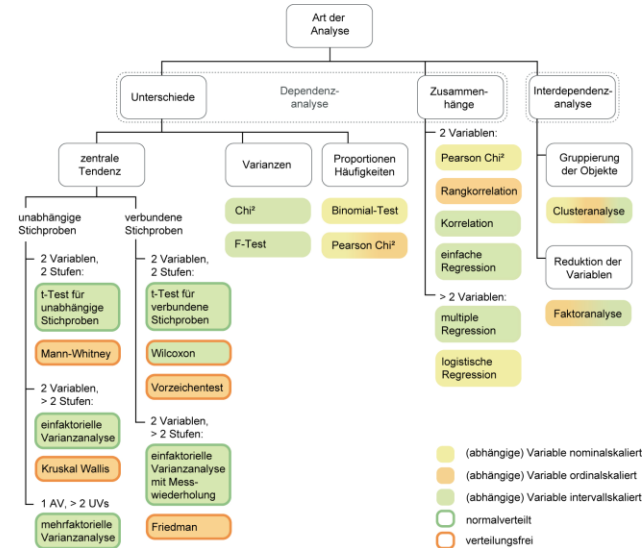


Image Credit:
<https://micheledamato.altervista.org/statistics-research-n-1>

Image credit: <https://www.methodenberatung.uzh.ch>

Recap: Scale of Measure – choice of machine learning model

The choice of a suitable machine learning model depends on the data types

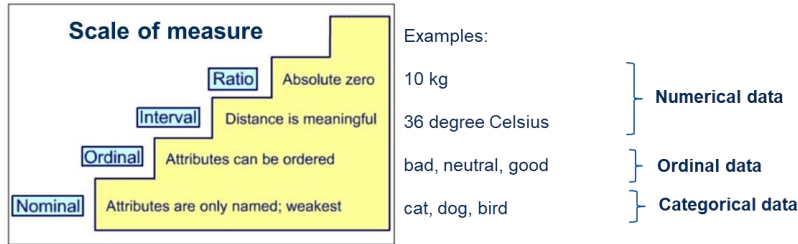


Image Credit:
<https://micheledamato.altervista.org/statistics-research-n-1>

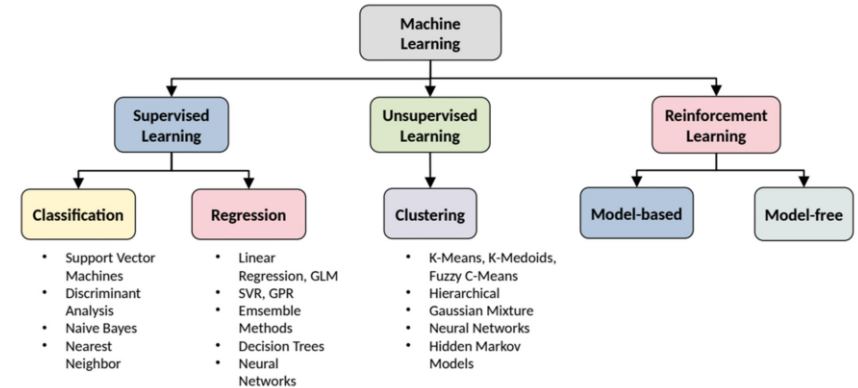


Image credit: <https://www.mdpi.com/2071-1050/13/9/5248>

Data Cleansing: Missing Values – definition

rooms_area_price_raw	address_raw	price_raw	description_raw	text_raw	rooms	area	price
2,5 Zimmer, Preis auf Anfrage	Wolserstrasse 30a, 8912 Obfelden, ZH	Preis auf Anfrage	«2.5 Zimmer Neubauwohnung im DG»	premium2,5 Zimmer, Preis auf AnfrageWolserstra...	2.5	<NA>	<NA>
2,5 Zimmer, 77 m², Preis auf Anfrage	Birchstrasse 185, 8050 Zürich, ZH	Preis auf Anfrage	«Lifestyle mit kühler Eleganz»	2,5 Zimmer, 77 m², Preis auf AnfrageBirchstras...	2.5	77	<NA>
4,5 Zimmer, 87 m², Preis auf Anfrage	Rütlistr. 15, 8307 Effretikon, ZH	Preis auf Anfrage	«Zentrale Wohnung in familiärer Umgebung zu ve...	4,5 Zimmer, 87 m², Preis auf AnfrageRütlistr. ...	4.5	87	<NA>
4,5 Zimmer, 104 m², Preis auf Anfrage	Neu Letten 5a, 8192 Zweidlen, ZH	Preis auf Anfrage	«4 ½ Zimmer-Dachwohnung, 8192 Zweidlen»	top4,5 Zimmer, 104 m², Preis auf AnfrageNeu Le...	4.5	104	<NA>
4,5 Zimmer, 132 m², Preis auf Anfrage	Hagenholzstrasse 55, 8050 Zürich, ZH	Preis auf Anfrage	«Ihr neues Zuhause im Tower55 - Oerlikon»	4,5 Zimmer, 132 m², Preis auf AnfrageHagenholz...	4.5	132	<NA>

Missing values are values that are not recorded for a variable for the observation in question. Missing values can reduce statistical power, cause bias in the estimation of the parameters, and lessen the representativeness of the samples.

Example: **missing values** in the variables «area» and «price» of the rental apartment data set.

Data Cleansing: Missing Values – imputation

Imputation: replacement of missing values

	col1	col2	col3	col4	col5			col1	col2	col3	col4	col5	
0	2	5.0	3.0	6	NaN	mean()		0	2.0	5.0	3.0	6.0	7.0
1	9	NaN	9.0	0	7.0			1	9.0	11.0	9.0	0.0	7.0
2	19	17.0	NaN	9	NaN			2	19.0	17.0	6.0	9.0	7.0

Mean of variables (columns) imputed to replace missing values.

When dealing with missing values, there are generally two methods available: **a) imputation** or **b) removal of data**. The imputation method is used to predict and replace missing values.

Data Cleansing: Missing Values – removal

Removal of missing values

rooms_area_price_raw	address_raw	price_raw	description_raw	text_raw	rooms	area	price
2.5 Zimmer, Preis auf Anfrage	Wolserstrasse 30a, 8912 Obfelden, ZH	Preis auf Anfrage	«2.5 Zimmer Neubauwohnung im DG»	premium2.5 Zimmer, Preis auf AnfrageWolserstra...	2.5	<NA>	<NA>
2.5 Zimmer, 77 m², Preis auf Anfrage	Birchstrasse 185, 8050 Zürich, ZH	Preis auf Anfrage	«Lifestyle mit kühler Eleganz»	2.5 Zimmer, 77 m², Preis auf AnfrageBirchstras...	2.5	77	<NA>
4.5 Zimmer, 87 m², Preis auf Anfrage	Rütlistr. 15, 8307 Effretikon, ZH	Preis auf Anfrage	«Zentrale Wohnung in familiärer Umgebung zu ve...	4.5 Zimmer, 87 m², Preis auf AnfrageRütlistr...	4.5	87	<NA>
4.5 Zimmer, 104 m², Preis auf Anfrage	Neu Letten 5a, 8192 Zweisiden, ZH	Preis auf Anfrage	«4 ½ Zimmer-Dachwohnung, 8192 Zweisiden»	top4.5 Zimmer, 104 m², Preis auf AnfrageNeu Le...	4.5	104	<NA>
4.5 Zimmer, 132 m², Preis auf Anfrage	Hagenholzstrasse 55, 8050 Zürich, ZH	Preis auf Anfrage	«Ihr neues Zuhause im Tower55 - Oerlikon»	4.5 Zimmer, 132 m², Preis auf AnfrageHagenholz...	4.5	132	<NA>

```
1 # Count missing values
2 print(pd.isna(df).sum(), '\n')
3
4 # Remove missing values
5 print(pd.isna(df.dropna()).sum())
```

```
web-scraper-order      0
web-scraper-start-url  0
rooms_area_price_raw   0
address_raw            0
price_raw              0
description_raw        0
text_raw              0
rooms                 86
area                  8
price                 8
dtype: int64

web-scraper-order      0
web-scraper-start-url  0
rooms_area_price_raw   0
address_raw            0
price_raw              0
description_raw        0
text_raw              0
rooms                 0
area                  0
price                 0
dtype: int64
```

The other option when dealing with missing values is to **remove data**. Note that removing data may not be the best option if there are not enough observations to result in a reliable analysis.

In Python, missing values can be identified using the **.isna()** and removed using the **.dropna()** method from the pandas library.

Data Cleansing: Missing Values – MCAR, MAR and MNAR

MCAR: Missing completely at random

- Missing is neither dependent on the actual variable nor on other variables
- Missing values can be removed

MAR: Missing at random

- Missing is independent of the variable itself where the value is missing
- Missing may depend on other variables in the data set
- If removed, the result is biased, the relationships between the parameters (e.g. in linear regression) remain unbiased
- **Example:** If older people are more likely to skip a specific survey question than younger people, the missingness mechanism is based on age, a different variable.

MNAR: Missing not at random

- Missing depends on the missing variable itself
- Missing may depend on other variables in the data set
- If removed, the result will be biased, and also the result of relationships between parameters
- **Example:** Poor people are less likely to state their income as many of them feel ashamed because they are poor.

The removal of missing values and its potential impact on modeling results.

Source: <https://statistik-und-beratung.de>

Data Cleansing: Duplicated Values

Identify duplicated values

```
DataFrame:
  product  price  sku department
0    fork    5.0  111    kitchen
1    spoon    5.5  112    kitchen
2    knife    6.0  113    kitchen
3    spoon    5.5  112    kitchen
4    fork    5.0  111    kitchen
5  spatula    7.5  114    kitchen
```

```
.duplicated Output:
```

```
0    True
1    True
2    False
3    True
4    True
5    False
```

```
dtype: bool
```

```
df.duplicated(keep=False)
```

Remove duplicated values

```
  product  price  sku department
0    fork    5.0  111    kitchen
1    spoon    5.5  112    kitchen
2    knife    6.0  113    kitchen
3    spoon    5.5  112    kitchen
4    fork    5.0  111    kitchen
5  spatula    7.5  114    kitchen
```

```
  product  price  sku department
0    fork    5.0  111    kitchen
1    spoon    5.5  112    kitchen
2    knife    6.0  113    kitchen
5  spatula    7.5  114    kitchen
```

```
df.drop_duplicates()
```

A **duplicated value** is one in which all values in at least one row of the data set are identical to all values in another row of that data set.

Duplicate values can cause bias and efficiency loss in statistical estimates (e.g. regression analysis) and should therefore be removed.

Image credit: <https://blog.hubspot.com/website/duplicated-pandas>

Data Cleansing: Outliers & Measurement Errors

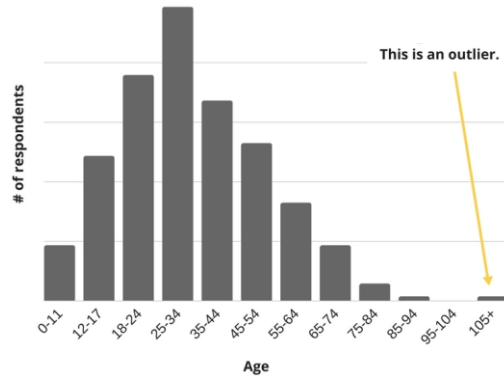
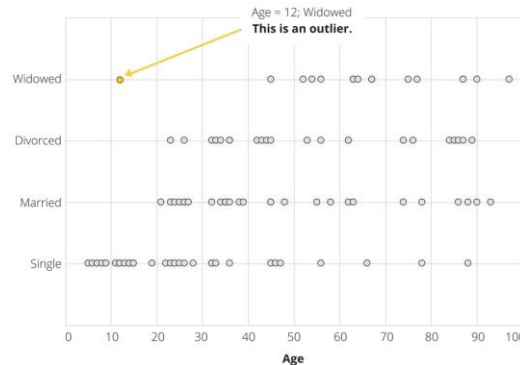


Image credit:
<https://humansofdata.atlan.com/2018/03/when-delete-outliers-dataset/>



An **outlier** is an observation that differs significantly from other observations. An outlier may be due to variability in the measurement, or it may indicate **measurement error**. Outliers can cause serious problems in statistical analyses and should be examined and only removed if they are errors.

String Manipulation: regular expressions

REGULAR EXPRESSION

`/[0-9]+/`

TEST STRING

My apartment costs \$1650.- per Month.
The next supermarket is 850m away.
We will meet us at 8:00 p.m.

MATCH INFORMATION

Match 1	22-26	1650
Match 2	64-67	850
Match 3	94-95	8
Match 4	96-98	00

A **regular expression (regex)** is a sequence of characters that specifies a search pattern in text. Often such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation.

Example tested on: <https://regex101.com>

String Manipulation: regex examples

REGULAR EXPRESSION
<code>/[^\i*&2@]</code>
TEST STRING
<code>My email is peter@gmail.com</code>

matches any character other than the letter *i*, an asterisk, ampersand, 2, or @ sign.

REGULAR EXPRESSION
<code>/^dog</code>
TEST STRING
<code>my pet is a dog.</code> <code>dogs are nice pets.</code>

begins with «dog»

REGULAR EXPRESSION
<code>/\s</code>
TEST STRING
<code>I hope the course is interesting!</code>

any whitespace character.

Examples tested on: <https://regex101.com>

String Manipulation: regex character classes

Regex Character Classes	
Regex	Usage
\d	Matches any digit
\D	Matches any non-digit
\w	Matches any alphanumeric character (incl. the underscore '_' character)
\W	Matches any non-alphanumeric character
\s	Matches any whitespace character
\S	Matches any non-whitespace character
.	Matches any character
[a-z]	Matches any lowercase character from 'a' to 'z'
[A-Z]	Matches any uppercase character from 'A' to 'Z'
[0-9]	Matches any digit from 0 to 9, equivalent with \d

The table shows frequently used **regular expressions** that can be used to specify a search pattern.

Image credit: <https://towardsdatascience.com/beginners-guide-to-regular-expressions-in-python-d16d2fa31587>

String Manipulation: extracting prices with regex

Extract rental price

```
1 import re
2
3 # Extract values from 'price_raw' strings
4 price = []
5 for i in df['price_raw']:
6     d1 = re.findall('[0-9]+', i)
7     try:
8         d2 = d1[0].strip()
9     except:
10         d2 = None
11     price.append(d2)
12
13 # Save as new variable in the pandas data frame
14 df['price'] = pd.Series(price, dtype="Int64")
15
16 # Print first 5 values of the list
17 print(df['price_raw'].head())
18 print(df['price'].head())
```

```
0    CHF 3180.-
1    CHF 3760.-
2    CHF 2860.-
3    CHF 2165.-
4    CHF 6900.-
Name: price_raw, dtype: object
0    3180
1    3760
2    2860
3    2165
4    6900
Name: price, dtype: Int64
```

price_raw = str

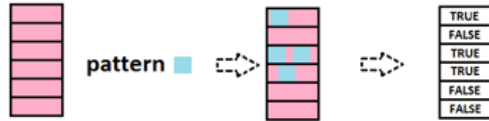
price = int

Python **regex** example to
extract numerical values of
rental apartment prices from
strings. The regex used here
is **[0-9]+**

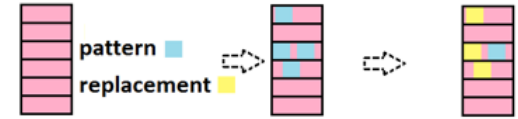
String Manipulation: overview of string functions

String manipulation (example functions)

Detect matches



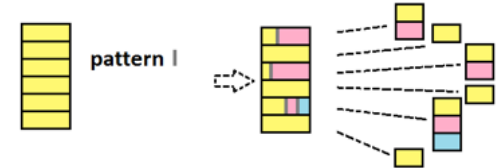
Mutate strings



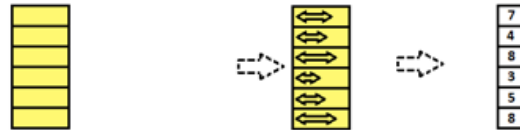
Subset strings



Split strings



Manage length



Count patterns

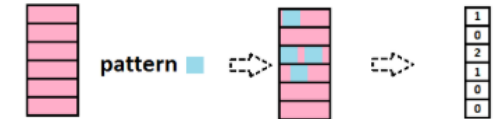


Image credit: https://rworkshop.uni.lu/lectures/lecture03_stringr.html#9

String Manipulation: letter case and white space

String manipulation (manage letter case and white space)

```
fruit = pd.Series(["appLE  ", "BaNana", "  PEAr", "PineaPPle"])
```

To lower case

```
list(fruit.str.lower())  
['apple ', 'banana', ' pear', 'pineapple']
```

To upper case

```
list(fruit.str.upper())  
['APPLE ', 'BANANA', ' PEAR', 'PINEAPPLE']
```

Capitalize

```
list(fruit.str.capitalize())  
['Apple ', 'Banana', ' pear', 'Pineapple']
```

Remove leading space

```
list(fruit.str.lstrip())  
['appLE ', 'BaNana', 'PEAr', 'PineaPPle']
```

Remove trailing space

```
list(fruit.str.rstrip())  
['appLE', 'BaNana', '  PEAr', 'PineaPPle']
```

Remove leading and trailing space

```
list(fruit.str.strip())  
['appLE', 'BaNana', 'PEAr', 'PineaPPle']
```

String Manipulation: pandas string functions

String manipulation (example functions)

```
fruit = pd.Series(["Apple", "Banana", "Pear", "Pineapple"])
```

Detect matches

```
list(fruit.str.contains('pp'))  
[True, False, False, True]
```

Subset strings

```
list(fruit[fruit.str.contains('pp')])  
['Apple', 'Pineapple']
```

Manage length

```
list(fruit.str.len())  
[5, 6, 4, 9]
```

Mutate strings

```
list(fruit.str.replace('pp', 'xx'))  
['Axxle', 'Banana', 'Pear', 'Pineaxxle']
```

Split strings

```
fruit.str.split('a')[1]  
['B', 'n', 'n', '']
```

Count patterns

```
list(fruit.str.count('p'))  
[2, 0, 0, 2]
```

String Manipulation: converting to upper case

Change strings in 'description_raw' and 'text_raw' to uppercase

```
1 # Change strings to uppercase
2 df['description_raw'] = df['description_raw'].str.upper()
3 print(df['description_raw'].head(10), '\n')
4
5 df['text_raw'] = df['text_raw'].str.upper()
6 print(df['text_raw'].head(10))
```

```
0 «GROSSE GALERIE, TERRASSE MIT PERGOLA, BERG- U...
1 «WUNDERSCHÖNE WOHNUNG IM ENGE-QUARTIER»
2 «WOHNMOMENTE ZUM FESTHALTEN»
3 «3,5 PIÈCES, 74 M²»
4 «WOHNANLAGE IM PARKRING - EXKLUSIVE WOHNUNG ZU...
5 «OHNE MIETKAUTION - SUPER RUHIGE LÄNDLICHE LAGE»
6 «ERSTVERMIETUNG AM ZÜRICHBERG: CHARMANTE 2.5-Z...
7 «ERSTVERMIETUNG NACH TOTAL-SANIERUNG: 3-ZIMMER...
8 «NEUES ZUHAUSE FÜR SIE UND IHRE KINDER»
9 «MODERN, HELL MIT PANORAMASICHT ÜBERS GLATTAL»
Name: description_raw, dtype: object
```

```
0 3,5 ZIMMER, 122 M², CHF 3180.-SUNNENBERGSTRASS...
1 2,5 ZIMMER, 78 M², CHF 3760.-LAVATERSTR. 63, 8...
2 5,5 ZIMMER, 115 M², CHF 2860.-LANGFURRENSTRASS...
3 TOP3,5 ZIMMER, 74 M², CHF 2165.-SANDBUCKWEG 5A...
4 5,5 ZIMMER, 195 M², CHF 6900.-PARKRING 59, 800...
5 2 ZIMMER, 47 M², CHF 1400.-IFANGWEG 1, 8610 US...
6 2,5 ZIMMER, 59 M², CHF 2920.-FLOBOTSTRASSE 2, ...
7 3 ZIMMER, 75 M², CHF 3003.-HINTERBERGSTRASSE 5...
8 4,5 ZIMMER, 97 M², CHF 1770.-FROHBERGSTR. 7, 8...
9 4,5 ZIMMER, 124 M², CHF 4460.-8306 BRÜTTISELLE...
Name: text_raw, dtype: object
```

In this example the variables `description_raw` and `text_raw` were set to **upper case**. This is useful because the text is often not consistently written in lowercase or uppercase letters.

String Manipulation: creating a binary variable

Create new binary (0/1) variable 'luxurious'

```
1 # Create a pattern which can be used to search the variable 'description_raw'
2 pattern = '(LOFT)|(SEESICHT)|(ATTIKA)|(LUXURIÖS)|(POOL)|(EXKLUSIV)'
3
4 # Create new variable 'luxurious' as binary dummy (0/1) variable
5 df['luxurious'] = df['description_raw'].str.contains(pat = pattern).astype(int)
6 print(df['luxurious'].sum())
7
8 # Show values
9 df[['description_raw', 'rooms', 'area', 'price', 'luxurious']]
```

71

	description_raw	rooms	area	price	luxurious
0	«GROSSE GALERIE, TERRASSE MIT PERGOLA, BERG- U...	3.5	122	3180	1
1	«WUNDERSCHÖNE WOHNUNG IM ENGE-QUARTIER»	2.5	78	3760	0
2	«WOHNMOMENTE ZUM FESTHALTEN»	5.5	115	2860	0
3	«3,5 PIÈCES, 74 M²»	3.5	74	2165	0
4	«WOHNANLAGE IM PARKRING - EXKLUSIVE WOHNUNG ZU...	5.5	195	6900	1
...	...	...	...	...	...
1003	«TRAUMHAFTE LOFTWOHNUNG GESUCHT?»	1.5	65	2470	1
1004	«AN SONNIGER LAGE, RICHTUNG WALD»	3.5	70	1465	0
1005	«IHRE WOHNUNG AN RUHIGER LAGE - BEFRISTETES WO...	2.5	56	1870	0
1006	«MODERNE 3.5-ZIMMERWOHNUNG IN ALTSTETTEN»	3.5	70	2190	0
1007	«SCHÖNE 3.5-ZIMMER-MAISONETTE-WOHNUNG MIT SONN...	3.5	96	2260	0

1008 rows × 5 columns

In this example the variable "luxurious" was created using the **.str.contains()** method which compares a string search pattern with the apartment descriptions. A value of '1' is assigned if one of the terms in 'pattern' occurs in the apartment descriptions of each apartment.

Date Formatting: time series data

Microsoft

377,25 €

↑199,50 % +251,29 5 J.

3. Sept., 14:14:28 UTC+2 · EUR · ETR · Haftungsausschluss

1 T. 5 T. 1 M. 6 M. YTD 1 J. 5 J. MAX

Wichtige Ereignisse >



Image credit: <https://www.google.com/finance>

Many data points contain temporal information such as the time of measurement. Such data are also referred to as **time series data**. A time series is a series of data points listed or visualized in time order.

Date Formatting: the datetime module

```
import datetime

# Current datetime
x = datetime.datetime.now()
print(x)
2022-09-06 10:16:38.728698

# Year format
print(x.strftime("%Y"))
2022

# Year-Month-Day format
print(x.strftime("%Y-%m-%d"))
2022-09-06

# Year-Month-Day Hour:Minute:Second format
print(x.strftime("%Y-%m-%d %H:%M:%S"))
2022-09-06 10:16:38
```

In order to deal with time series data, such data must be brought into specific date formats. In **Python** a module named **datetime** can be used to work with dates in the form of datetime objects. Datetime objects are not only useful when working with time series data but also to assign the time of data measurement / preparation.

Date Formatting: datetime in a data frame

Including current datetime

```
1 import datetime
2
3 # Get and format datetime
4 df['datetime'] = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
5
6 # Show values
7 df[['description_raw', 'rooms', 'area', 'price', 'luxurious', 'price_per_m2', 'datetime']]
```

	description_raw	rooms	area	price	luxurious	price_per_m2	datetime
0	«GROSSE GALERIE, TERRASSE MIT PERGOLA, BERG- U...	3.5	122	3180	1	26.07	2022-09-07 08:41:49
1	«WUNDERSCHÖNE WOHNUNG IM ENGE-QUARTIER»	2.5	78	3760	0	48.21	2022-09-07 08:41:49
2	«WOHNMONENTE ZUM FESTHALTEN»	5.5	115	2860	0	24.87	2022-09-07 08:41:49
3	«3,5 PIÈCES, 74 M²»	3.5	74	2165	0	29.26	2022-09-07 08:41:49
4	«WOHNANLAGE IM PARKRING - EXKLUSIVE WOHNUNG ZU...	5.5	195	6900	1	35.38	2022-09-07 08:41:49
...	...	...	...	...	...	...	...
1003	«TRAUMHAFTE LOFTWOHNUNG GESUCHT?»	1.5	65	2470	1	38.0	2022-09-07 08:41:49
1004	«AN SONNIGER LAGE, RICHTUNG WALD»	3.5	70	1465	0	20.93	2022-09-07 08:41:49
1005	«IHRE WOHNUNG AN RUHIGER LAGE - BEFRISTETES WO...	2.5	56	1870	0	33.39	2022-09-07 08:41:49
1006	«MODERNE 3.5-ZIMMERWOHNUNG IN ALTSTETTEN»	3.5	70	2190	0	31.29	2022-09-07 08:41:49
1007	«SCHÖNE 3.5-ZIMMER-MAISONETTE-WOHNUNG MIT SONN...	3.5	96	2260	0	23.54	2022-09-07 08:41:49

1008 rows × 7 columns

Python example of
including and formatting
datetime as variable in a
data frame.

Data Discretization: binning with pandas

```
import pandas as pd

# Sample data
data = pd.DataFrame({
    'Age': [5, 12, 17, 24, 29, 35, 45, 52, 68, 72]
})

# Fixed-width binning
bins = [0, 10, 20, 30, 40, 50, 60, 70, 80]
labels = ['0-10', '11-20', '21-30', '31-40', '41-50', '51-60', '61-70', '71-80']
data['Age_Bin'] = pd.cut(data['Age'], bins=bins, labels=labels)

# Output the binned data
print(data)
```

	Age	Age_Bin
0	5	0-10
1	12	11-20
2	17	11-20
3	24	21-30
4	29	21-30
5	35	31-40
6	45	41-50
7	52	51-60
8	68	61-70
9	72	71-80

Example using pandas.cut

Data Discretization (also called “Data Binning”) is a technique that involves dividing a continuous variable into a finite number of intervals or “bins”. These bins are then treated as categorical variables. Data binning helps to reduce the effects of minor observation errors and helps in managing noise in data, especially when the data is too granular or detailed.

Data Discretization: KBinsDiscretizer, pros and cons

```
from sklearn.preprocessing import KBinsDiscretizer

# Sample data
data = [[5], [12], [17], [24], [29], [35], [45], [52], [68], [72]]

# Binning into 3 equal-sized bins
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
binned_data = est.fit_transform(data)

print(binned_data)
```

```
[[0.]
 [0.]
 [0.]
 [0.]
 [1.]
 [1.]
 [1.]
 [2.]
 [2.]
 [2.]]
```

Example using `sklearn.preprocessing.KBinsDiscretizer`

Advantages:

- **Reduces Noise:** By grouping data, it smooths the dataset and reduces the impact of minor variations and noise.
- **Improves Interpretability:** Binning can make data easier to understand and interpret, particularly for end-users who may not be familiar with continuous variables.
- **Enhances Feature Engineering:** Binning can create meaningful categorical features that might improve the performance of certain models, particularly those that work better with categorical inputs (e.g., decision trees).

Disadvantages:

- **Loss of Information:** Discretization can result in a loss of information, as the exact values of the continuous variables are replaced by the bin categories.
- **Choosing Bins:** The selection of bin edges can be arbitrary and might require domain knowledge to be effective.

One-Hot Encoding: concept

Example of One-Hot Encoding:

Suppose you have a dataset with a "Color" column:

Color
Red
Green
Blue
Green
Red

After applying One-Hot Encoding, the dataset would look like this:

Color_Red	Color_Green	Color_Blue
1	0	0
0	1	0
0	0	1
0	1	0
1	0	0

One-Hot Encoding is a technique that converts categorical variables into a format that can be provided to machine learning algorithms. Specifically, it transforms categorical data, which can have multiple categories, into vectors with binary values.

One-Hot Encoding: OneHotEncoder, pros and cons

```
from sklearn.preprocessing import OneHotEncoder

# Sample data
data = [['Red'], ['Green'], ['Blue'], ['Green'], ['Red']]

# One-Hot Encoder
encoder = OneHotEncoder(sparse=False)
one_hot_encoded_data = encoder.fit_transform(data)

print(one_hot_encoded_data)
```

```
[[0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Example using `sklearn.preprocessing.OneHotEncoder`

Note: since scikit-learn 1.2 the parameter is called `sparse_output`; `sparse` was removed in version 1.4. Use `OneHotEncoder(sparse_output=False)`.

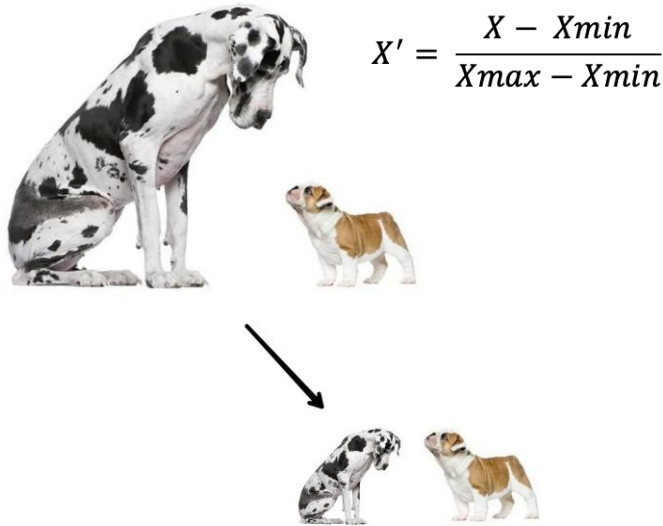
Advantages:

- **No Ordinal Relationships:** Prevents the model from assuming any kind of ordinal relationship between the categories, which is important when the categories are nominal.
- **Improved Algorithm Performance:** Algorithms like linear regression, logistic regression, and neural networks often perform better with one-hot encoded data because it allows them to treat each category as a separate feature.

Disadvantages:

- **Curse of Dimensionality:** Can lead to a large number of features, especially if the categorical variable has many categories. This can result in high memory usage and slow computation.
- **Sparse Data:** Many of the entries in the one-hot encoded matrix are zeros, which can result in sparse data, potentially leading to inefficient processing and storage.

Data Scaling: introduction



Data scaling is a preprocessing step in machine learning and data analysis that involves adjusting the values of numerical data to a common scale, without distorting differences in the ranges of values. Its primary purpose is to bring all the features into a comparable range, which can significantly improve the performance of machine learning algorithms, especially those that are sensitive to the magnitude of the data like linear & logistic regression and k-nearest neighbors.

Image credit: <https://medium.com/swlh/data-normalisation-with-r-6ef1d1947970>

Data Scaling: Min-Max Scaling

Common Methods of Data Scaling: Min-Max Scaling

Formula: The Min-Max Scaling formula is:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Here:

- X is the original value.
- $X_{\min}$ is the minimum value of the feature in the dataset.
- $X_{\max}$ is the maximum value of the feature in the dataset.
- X_{scaled} is the transformed value.

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler=MinMaxScaler()
```

Min-Max Scaling is a method used to rescale the values of data features so that they fit within a specific range, typically between 0 and 1. This method transforms the data by adjusting the minimum and maximum values of the feature to the desired range.

Data Scaling: Max Absolute Scaling

Common Methods of Data Scaling: Max Absolute Scaling

Formula: The formula for Max Abs Scaling is:

$$X_{\text{scaled}} = \frac{X}{|X_{\text{max}}|}$$

Here:

- X is the original value.
- X_{max} is the maximum absolute value of the feature.
- X_{scaled} is the scaled value.

```
from sklearn.preprocessing import MaxAbsScaler
```

```
scaler=MaxAbsScaler()
```

Max Absolute Scaling is a data scaling technique that adjusts the values of features by dividing each value by the maximum absolute value of that feature. This method ensures that all the scaled values fall within the range of -1 to 1. Unlike some other scaling methods, Max Abs Scaling does not shift the data (i.e., it does not center it around zero); instead, it preserves the original distribution and sparsity of the data.

Data Scaling: Robust Scaling

Common Methods of Data Scaling: Robust Scaling

Formula: The formula for Robust Scaling is:

$$X_{\text{scaled}} = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

Here:

- X is the original value.
- $\text{median}(X)$ is the median of the feature in the dataset.
- $\text{IQR}(X)$ is the interquartile range of the feature, calculated as the difference between the 75th percentile and the 25th percentile.

```
from sklearn.preprocessing import RobustScaler
```

```
scaler=RobustScaler()
```

Robust Scaling is a method of scaling data that is particularly effective when the dataset contains outliers. Unlike other scaling methods that might be affected by extreme values, Robust Scaling uses the median and the interquartile range (IQR) of the data to rescale the features. This approach reduces the influence of outliers and focuses on the central tendency of the data.

Data Standardization (Z-score Normalization)

Formula: The formula for standardization is:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

Here:

- X is the original value.
- μ is the mean of the feature.
- σ is the standard deviation of the feature.
- X_{scaled} is the standardized value.

```
from sklearn.preprocessing import StandardScaler
```

```
sc=StandardScaler()
```

Data Standardization, often referred to as **Z-score Normalization**, is a method of scaling data that adjusts the values of features so that they have a mean of 0 and a standard deviation of 1. The values are then centered around 0 with unit variance; the shape of the distribution does not change (standardization does not make data normally distributed).

Data Transformation (Statistics): definition

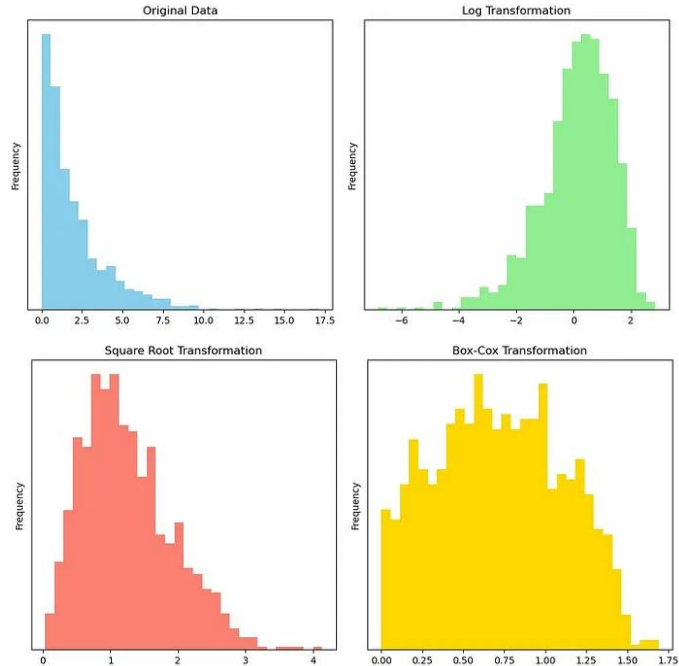


Image credit: <https://neverforget-1975.medium.com/embracing-data-transformation-in-statistical-analysis-9a85d953b3cb>

In statistics, **Data Transformation** involves applying mathematical functions to data to meet the assumptions of statistical models, improve interpretability, or enhance the robustness of analysis.

Data Transformation (Statistics): common transformations

Common Statistical Transformations:

- Logarithmic Transformation (log): $Y' = \log(Y)$
- Square Root Transformation: $Y' = \sqrt{Y}$
- Inverse Transformation (Reciprocal): $Y' = \frac{1}{Y}$
- Exponential Transformation: $Y' = e^Y$
- Box-Cox Transformation: A parameterized family of transformations defined as $Y' = \frac{(Y^\lambda - 1)}{\lambda}$, where λ is the transformation parameter.

Note: the Box-Cox transformation requires $Y > 0$; for $\lambda = 0$ it is defined as $Y' = \log(Y)$.

Data Transformation (Statistics): relationship between x and y

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import boxcox

# Generate some data points
np.random.seed(0)
x = np.linspace(1, 10, 100)
# Exponential relationship (nonlinear)
y = 2 * np.exp(0.5 * x) + np.random.normal(0, 2, size=x.shape)

# Ensure y is positive for Box-Cox (Box-Cox requires strictly positive values)
y = y - np.min(y) + 1

# Create the figure and axes
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# Scatter plot of original nonlinear relationship
ax1.scatter(x, y, color='green', s=20)
ax1.set_title('Original Relationship (Nonlinear)')
ax1.set_xlabel('x')
ax1.set_ylabel('y')

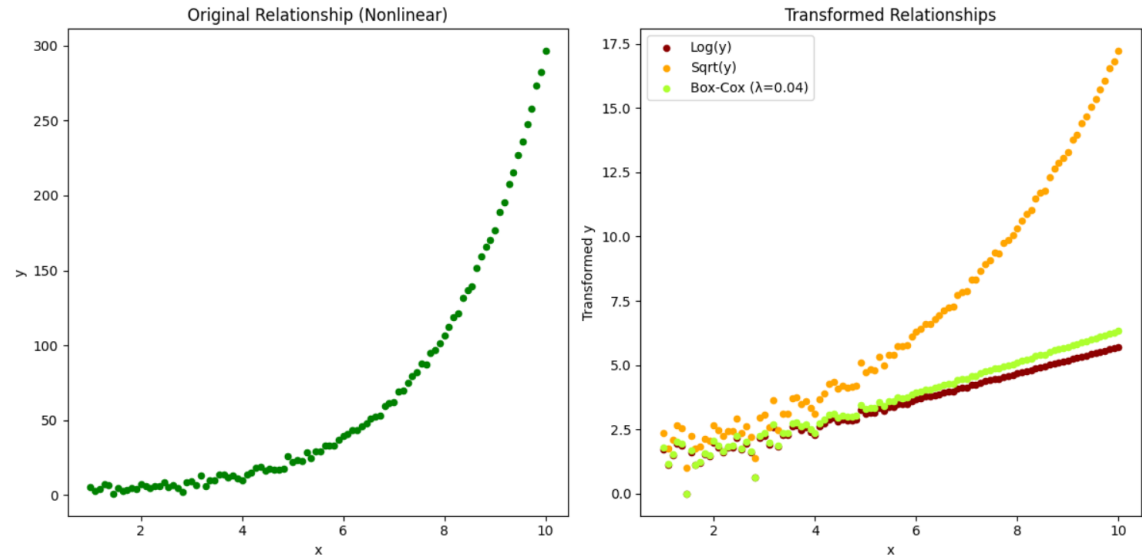
# Apply transformations
log_y = np.log(y)      # log transformation
sqrt_y = np.sqrt(y)     # Square-root transformation
y_boxcox, lambda_opt = boxcox(y) # Box-Cox transformation

# Scatter plot of transformed relationships
ax2.scatter(x, log_y, color='darkred', label='Log(y)', s=20)
ax2.scatter(x, sqrt_y, color='orange', label='Sqrt(y)', s=20)
ax2.scatter(x, y_boxcox, color='greenyellow', label=f'Box-Cox ( $\lambda={lambda_opt:.2f}$ )', s=20)

ax2.set_title('Transformed Relationships')
ax2.set_xlabel('x')
ax2.set_ylabel('Transformed y')
ax2.legend()

# Show the plots
plt.tight_layout()
plt.show()
```

Relationship between x and y before and after **transformation**



Data Transformation (Statistics): effect on residuals

Effect of log-transformed target variable on the residuals of a linear regression

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress

# Generate some data points
# np.random.seed(0)
x = np.linspace(1, 10, 100)
# Modify the exponential relationship to grow more slowly and reduce noise
y = 2 * np.exp(0.3 * x) + np.random.normal(0, 1, size=x.shape)

# Perform linear regression on the original data
slope_original, intercept_original, _, _ = linregress(x, y)
y_pred_original = slope_original * x + intercept_original
residuals_original = y - y_pred_original

# Log transformation of y
log_y = np.log(y)

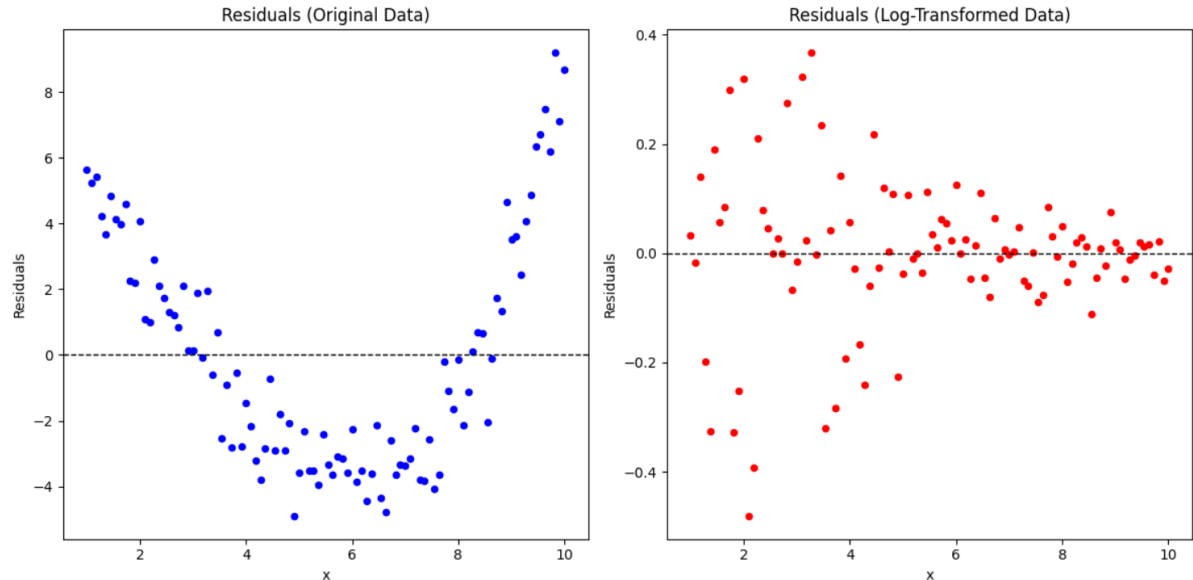
# Perform linear regression on the log-transformed data
slope_log, intercept_log, _, _ = linregress(x, log_y)
y_pred_log = slope_log * x + intercept_log
residuals_log = log_y - y_pred_log

# Create the figure and axes for residuals comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# Plot residuals for original data
ax1.scatter(x, residuals_original, color='blue', s=20)
ax1.axhline(0, color='black', linestyle='--', linewidth=1)
ax1.set_title('Residuals (Original Data)')
ax1.set_xlabel('x')
ax1.set_ylabel('Residuals')

# Plot residuals for log-transformed data
ax2.scatter(x, residuals_log, color='red', s=20)
ax2.axhline(0, color='black', linestyle='--', linewidth=1)
ax2.set_title('Residuals (Log-Transformed Data)')
ax2.set_xlabel('x')
ax2.set_ylabel('Residuals')

# Show the plots
plt.tight_layout()
plt.show()
```



Combining & Organizing Data

Combining data

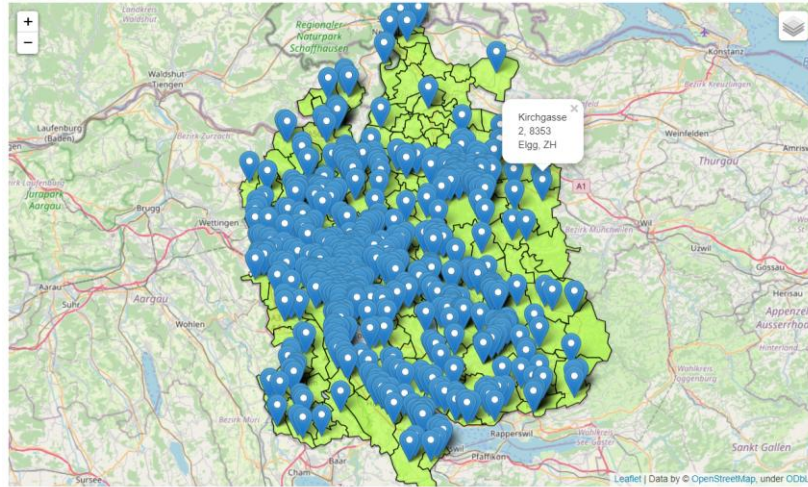
- `.merge()`
- `.join()`
- `.concat()`

Sorting data

Aggregation of data

Reshaping and pivoting tables

Combining data: example data

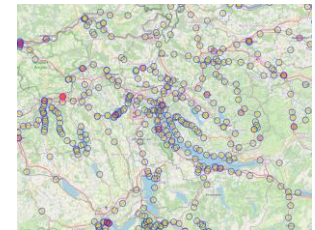
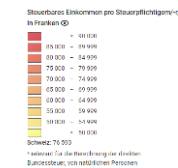
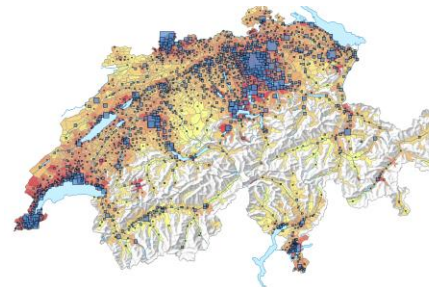


	web-scraper-order	address_raw	lat	lon	bfs_number	bfs_name
	0	1662023695-433	Sonnenbergstrasse 15, 8633 Wolfhausen, ZH	47.255714	8.804976	112 Bubikon
	24	1662023720-634	Blumenbergstrasse 7, 8633 Wolfhausen, ZH	47.254879	8.793746	112 Bubikon
	606	1662023745-834	8608 Bubikon, ZH	47.277386	8.800306	112 Bubikon
	759	1662023701-503	8608 Bubikon, ZH	47.277386	8.800306	112 Bubikon
	1	1662023745-820	Lavaterstr. 63, 8002 Zürich, ZH	47.361378	8.533339	261 Zürich

Example data to combine: rental apartment data & municipality characteristics like mean taxable income, population density, distance to nearest train station, ...



Gemeinde		Bevölkerung		Fläche		Dichte		Höhe		Entfernung		Verkehr		Wirtschaft		Sonstiges	
Code	Name	2010	2015	km²	km²	2010	2015	m	m	km	km	km	km	km	km	km	km
1	Basel-Stadt	199,000	201,000	141.0	141.0	1,414	1,426	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000



Combining data: techniques

Techniques to combine data

Join – Methods

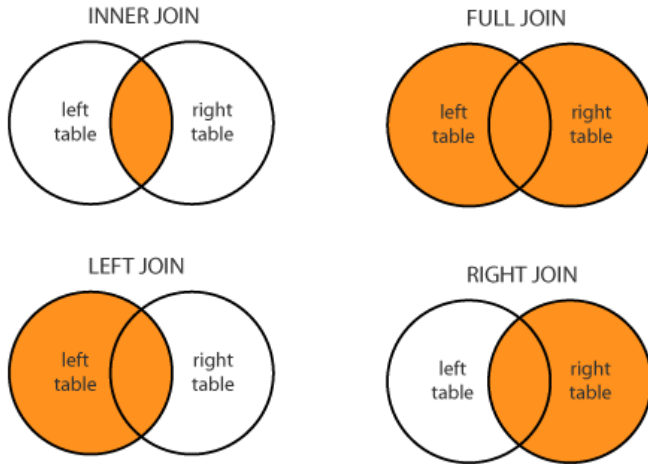


Image credit: <https://www.dofactory.com/sql/join>

Spatial Overlay Analysis

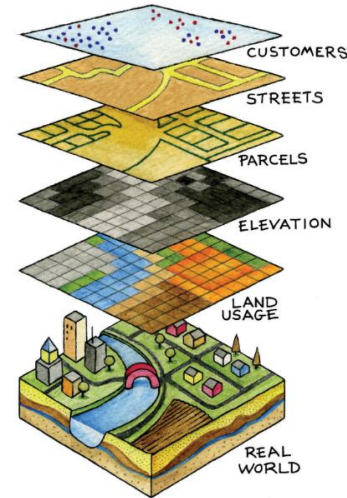


Image credit: <https://www.canvasgfx.com/en/support/product-support/what-is-gis/>

Combining data: pandas methods

- .merge()** for combining data on common columns or indices
- .join()** for combining data on a key column or an index
- .concat()** for combining data frames across rows or columns

Pandas can be used to combine data from different sources. This allows data to be enriched prior to analysis or modeling. The methods used are **merge**, **join** and **concatenate**.

Combining data: .merge()

```
result = pd.merge(left, right, on="key")
```

left				right				result					
	key	A	B		key	C	D		key	A	B	C	D
0	K0	A0	B0	0	K0	C0	D0	0	K0	A0	B0	C0	D0
1	K1	A1	B1	1	K1	C1	D1	1	K1	A1	B1	C1	D1
2	K2	A2	B2	2	K2	C2	D2	2	K2	A2	B2	C2	D2
3	K3	A3	B3	3	K3	C3	D3	3	K3	A3	B3	C3	D3

Pandas provides a single method, **.merge()**, as the entry point for all standard database join operations between data frame or named series objects.

Image credit: <https://pandas.pydata.org>

Combining data: .merge() example

	web-scraper-order	address_raw	datetime	rooms	area	luxurious	price	price_per_m2
0	1662023695-433	Sonnenbergstrasse 15, 8633 Wolfhausen, ZH	2022-09-07 09:00:00	3.5	122	1	3180	26.07
1	1662023745-820	Lavaterstr. 63, 8002 Zürich, ZH	2022-09-07 09:00:00	2.5	78	0	3760	48.21
2	1662023742-807	Langfurrenstrasse 5c, 8623 Wetzikon ZH, ZH	2022-09-07 09:00:00	5.5	115	0	2860	24.87
3	1662023804-1290	Sandbuckweg 5A, 8157 Dielsdorf, ZH	2022-09-07 09:00:00	3.5	74	0	2165	29.26
4	1662023739-771	Parking 59, 8002 Zürich, ZH	2022-09-07 09:00:00	5.5	195	1	6900	35.38

df

	web-scraper-order	address_raw	lat	lon	bfs_number	bfs_name
0	1662023695-433	Sonnenbergstrasse 15, 8633 Wolfhausen, ZH	47.255714	8.804976	112	Bubikon
1	1662023720-634	Blumenbergstrasse 7, 8633 Wolfhausen, ZH	47.254879	8.793746	112	Bubikon
2	1662023745-834	8608 Bubikon, ZH	47.277386	8.800306	112	Bubikon
3	1662023701-503	8608 Bubikon, ZH	47.277386	8.800306	112	Bubikon
4	1662023745-820	Lavaterstr. 63, 8002 Zürich, ZH	47.361378	8.533339	261	Zürich

df_geo

Join data geo-information to rental apartment data using .merge()

```
1 df2 = df.merge(df_geo[['web-scraper-order',
2                       'lat',
3                       'lon',
4                       'bfs_number',
5                       'bfs_name']],
6               on="web-scraper-order")
7 df2.head()
```

	web-scraper-order	address_raw	datetime	rooms	area	luxurious	price	price_per_m2	lat	lon	bfs_number	bfs_name
0	1662023695-433	Sonnenbergstrasse 15, 8633 Wolfhausen, ZH	2022-09-07 09:00:00	3.5	122	1	3180	26.07	47.255714	8.804976	112	Bubikon
1	1662023745-820	Lavaterstr. 63, 8002 Zürich, ZH	2022-09-07 09:00:00	2.5	78	0	3760	48.21	47.361378	8.533339	261	Zürich
2	1662023742-807	Langfurrenstrasse 5c, 8623 Wetzikon ZH, ZH	2022-09-07 09:00:00	5.5	115	0	2860	24.87	47.328632	8.810400	121	Wetzikon (ZH)
3	1662023804-1290	Sandbuckweg 5A, 8157 Dielsdorf, ZH	2022-09-07 09:00:00	3.5	74	0	2165	29.26	47.477493	8.456285	86	Dielsdorf
4	1662023739-771	Parking 59, 8002 Zürich, ZH	2022-09-07 09:00:00	5.5	195	1	6900	35.38	47.366898	8.528817	261	Zürich

.merge() example

Combining data: .join()

```
result = left.join(right, on="key")
```

left				right			result					
	A	B	key		C	D		A	B	key	C	D
0	A0	B0	K0				0	A0	B0	K0	C0	D0
1	A1	B1	K1	K0	C0	D0	1	A1	B1	K1	C1	D1
2	A2	B2	K0	K1	C1	D1	2	A2	B2	K0	C0	D0
3	A3	B3	K1				3	A3	B3	K1	C1	D1

Pandas' **.join()** is a convenient method for combining the columns of two potentially differently-indexed data frames into a single result data frame.

Image credit: <https://pandas.pydata.org>

Combining data: .concat()

```
result = pd.concat([df1, df2, df3])
```

df1					result				
	A	B	C	D		A	B	C	D
0	A0	B0	C0	D0	0	A0	B0	C0	D0
1	A1	B1	C1	D1	1	A1	B1	C1	D1
2	A2	B2	C2	D2	2	A2	B2	C2	D2
3	A3	B3	C3	D3	3	A3	B3	C3	D3
df2					4	A4	B4	C4	D4
	A	B	C	D	5	A5	B5	C5	D5
4	A4	B4	C4	D4	6	A6	B6	C6	D6
5	A5	B5	C5	D5	7	A7	B7	C7	D7
6	A6	B6	C6	D6	8	A8	B8	C8	D8
7	A7	B7	C7	D7	9	A9	B9	C9	D9
df3					10	A10	B10	C10	D10
	A	B	C	D	11	A11	B11	C11	D11
8	A8	B8	C8	D8					
9	A9	B9	C9	D9					
10	A10	B10	C10	D10					
11	A11	B11	C11	D11					

Pandas' **.concat()** method takes a list or dict of homogeneously-typed objects and concatenates them with some configurable handling of 'what to do with the other axes'.

Image credit: <https://pandas.pydata.org>

Sorting data: .sort_values()

```
import pandas as pd
import numpy as np

# Create data frame
df = pd.DataFrame({
    'col1': ['A', 'A', 'B', np.nan, 'D', 'C'],
    'col2': [2, 1, 9, 8, 7, 4],
    'col3': [0, 1, 9, 4, 2, 3],
    'col4': ['a', 'B', 'c', 'D', 'e', 'F']
})
df

# Sort data ascending by columns 'col1' & 'col2'
df.sort_values(by=['col1', 'col2'], ascending=True)
col1  col2  col3  col4
1     A     1     1     B
0     A     2     0     a
2     B     9     9     c
5     C     4     3     F
4     D     7     2     e
3  NaN     8     4     D
```

Values in a data frame can be sorted using the **.sort_values()** method.

Sorting data: .sort_values() example

Sorting data

```
1 # Sorting data by 'price' and 'area' with highest price above (ascending=False)
2 df3.sort_values(by=['price', 'area'],
3               ascending=False).head()
```

	web-scraper-order	address_raw	datetime	rooms	area	luxurious	price	price_per_m2	lat	lon	bfs_number	bfs_name	pop	pop_dens
76	1662023771-1030	Parking 57, 8002 Zürich, ZH	2022-09-07 09:00:00	5.0	187	1	8900	47.59	47.366734	8.528435	261	Zürich	420217	4778.994655
174	1662023689-385	8053 Zürich, ZH	2022-09-07 09:00:00	4.5	171	1	7900	46.20	47.358616	8.578999	261	Zürich	420217	4778.994655
36	1662023733-742	Giessereistrasse 12, 8005 Zürich, ZH	2022-09-07 09:00:00	2.5	282	0	7500	26.60	47.390221	8.518072	261	Zürich	420217	4778.994655
5	1662023739-771	Parking 59, 8002 Zürich, ZH	2022-09-07 09:00:00	5.5	195	1	6900	35.38	47.366898	8.528817	261	Zürich	420217	4778.994655
18	1662023786-1149	Freudenbergstrasse 94, 8044 Zürich, ZH	2022-09-07 09:00:00	4.5	128	0	6630	51.80	47.384132	8.556532	261	Zürich	420217	4778.994655

.sort_values()
example

Aggregation of data: .groupby()

```
import pandas as pd

# Create data frame
df = pd.DataFrame({'Animal': ['Falcon', 'Falcon',
                              'Parrot', 'Parrot'],
                  'Max Speed': [380., 370., 24., 26.]})

df

```

	Animal	Max Speed
0	Falcon	380.0
1	Falcon	370.0
2	Parrot	24.0
3	Parrot	26.0

```

# Use .groupby to calculate the mean of 'Max Speed' per group
df.groupby(['Animal']).mean()

```

	Max Speed
Animal	
Falcon	375.0
Parrot	25.0

The **groupby** operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Aggregation of data: .groupby() example

Aggregation of data

```
1 # Aggregation using .groupby()  
2 df3[['rooms', 'price']].groupby(['rooms']).mean().round(2)
```

price	
rooms	
1.0	1515.89
1.5	1946.31
2.0	1562.81
2.5	2273.56
3.0	1825.42
3.5	2339.69
4.0	2261.10
4.5	2650.76
5.0	3257.23
5.5	3013.31
6.0	4240.00
6.5	4760.00
9.0	4000.00

.groupby() example

Reshaping and pivoting tables: overview

`.stack()`

`.unstack()`

`.melt()`

`.pivot()`

`.pivot_table()`

Transforming or **reshaping** data into an understandable form is one of the key aspects in data analytics. Pandas provides various built-in methods for reshaping data frames. These are: **`.stack()`**, **`.unstack()`**, **`.melt()`**, **`.pivot()`** and **`.pivot_table()`**.

Reshaping and pivoting tables: `.stack()` and `.unstack()`

Stack

df2

		A	B
first	second		
bar	one	1	2
	two	3	4
baz	one	5	6
	two	7	8

MultiIndex

stacked = df2.stack()

first	second		
bar	one	A	1
		B	2
	two	A	3
		B	4
baz	one	A	5
		B	6
	two	A	7
		B	8

MultiIndex

Unstack

stacked

first	second		
bar	one	A	1
		B	2
	two	A	3
		B	4
baz	one	A	5
		B	6
	two	A	7
		B	8

MultiIndex

stacked.unstack()

		A	B
first	second		
bar	one	1	2
	two	3	4
baz	one	5	6
	two	7	8

MultiIndex

The methods `.stack()` and `.unstack()` are used for restructuring columns and rows (index). `.stack()`: stacks the prescribed level(s) from column to row. `.unstack()`: unstacks the prescribed level(s) from row to column.

Image credit: <https://pandas.pydata.org/pandas-docs/version/0.23.4/reshaping.html>

Reshaping and pivoting tables: .pivot()

Pivot

df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t



```
df.pivot(index='foo',  
          columns='bar',  
          values='baz')
```

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

The **.pivot()** method is used to reshape a given data frame organized by given index / column values.

Image credit: <https://pandas.pydata.org/pandas-docs/version/0.23.4/reshaping.html>

Reshaping and pivoting tables: .melt()

Melt

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

The **.melt()** method is useful to bring a data frame into a format where one or more columns are identifier variables (`id_vars`), while all other columns, considered measured variables (`value_vars`), are “unpivoted” to the row axis, leaving just two non-identifier columns, ‘variable’ and ‘value’.

Image credit: <https://pandas.pydata.org/pandas-docs/version/0.23.4/reshaping.html>

Reshaping and pivoting tables: .pivot_table()

Pivot 1

```
df_long.pivot_table(index=["student", "school"],  
                    columns="class",  
                    values="grade",  
                    reset_index=True)
```

class	student	school	english	math	physics
0	Andy	Z	10	20	30
1	Bernie	Y	100	200	300
2	Cindy	Z	1000	2000	3000
3	Deb	Y	10000	20000	30000

Pivot 2

```
df_long.pivot_table(index=["student", "school"],  
                    columns="class",  
                    values="grade",  
                    aggfunc='sum',  
                    margins=True,  
                    reset_index=True)
```

class	student	school	english	math	physics	All
0	Andy	Z	10	20	30	60
1	Bernie	Y	100	200	300	600
2	Cindy	Z	1000	2000	3000	6000
3	Deb	Y	10000	20000	30000	60000
4	All		11110	22220	33330	66660

Pivot 3

```
df_long.pivot_table(index=["student", "school"],  
                    # default aggfunc='mean')
```

student	school	grade
Andy	Z	20 $((10+20+30)/3)$
Bernie	Y	200 $((100+200+300)/3)$
Cindy	Z	2000 $((1000+2000+3000)/3)$
Deb	Y	20000 $((10000+20000+30000)/3)$

Pivot 4

```
df_long.pivot_table(index='student',
                    columns=['school', 'class'],
                    values='grade',
                    fill_value=-5)
```

The .pivot_table() method in pandas is used to reshape and summarize data from a data frame.

Image credit: <https://towardsdatascience.com/reshape-pandas-dataframe-with-pivot-table-in-python-tutorial-and-visualization-2248c2012a31>

Reshaping and pivoting tables: .pivot_table() example

Pivoting data using .pivot_table()

```
1 # Using pivot_table to reshape the data and calculate means
2 pd.pivot_table(df3[['rooms', 'price', 'area']],
3                 index=['rooms'],
4                 values=['price', 'area'],
5                 aggfunc=np.mean)
```

	area	price
rooms		
1.0	42.600000	1515.888889
1.5	61.200000	1946.311111
2.0	55.194444	1562.805556
2.5	68.704698	2273.563758
3.0	68.781250	1825.421875
3.5	87.240458	2339.687023
4.0	87.360000	2261.100000
4.5	110.764368	2650.764368
5.0	124.769231	3257.230769
5.5	140.068966	3013.310345
6.0	153.000000	4240.000000
6.5	194.000000	4760.000000
9.0	300.000000	4000.000000

index['rooms'] = index of data frame

values[['price', 'area']] = pivoted (aggregated) values

.pivot_table() example

Note: since pandas 2.1, passing np.mean as aggfunc raises a FutureWarning: pandas currently maps it to its built-in mean, but in a future version the callable will be applied directly. Pass the string 'mean' to keep the current behaviour.

Key Takeaways and Learning Objectives

You ...

- can perform data cleansing on the data
- know what missing values are and how to deal with them
- know what duplicated values are and how to deal with them
- can explain the terms outliers & measurement errors and know the differences
- can explain regular expressions and the string manipulation methods shown in this course
- can perform date formatting on the data
- can perform discretization, one-hot encoding, scaling, standardization, and transformation on the data
- can perform the different methods to combine pandas data frames like merge, join, concat
- can sort data with Python
- can aggregate data using the `.groupby()` method
- can apply the different methods to reshape data shown in this course
- can pivot data with Python
- can prepare, aggregate and organize data in Jupyter notebooks

Literature

Parker, J. R. (2021). Python: An Introduction to Programming. Mercury Learning and Information. Link: https://zhaw.swisscovery.sls.ch/permalink/41SLSP_ZAW/1ivdr18/alma99116849005705510

Navlani, A., Fandango, A., & Idris, I. (2021). Python Data Analysis: Perform data collection, data processing, wrangling, visualization, and model building using Python. Packt Publishing Ltd.

Pandas User Guide (n.d.). https://pandas.pydata.org/docs/user_guide

Python 3 Documentation (n.d.). <https://docs.python.org/3/>

Sauer, S., 2018. Moderne Datenanalyse mit R: Daten einlesen, aufbereiten, visualisieren und modellieren, 1. Auflage 2019. ed, FOM-Edition. Springer Fachmedien Wiesbaden GmbH, Wiesbaden.

Vanderplas, J.T., 2016. Python data science handbook: essential tools for working with data, First edition. ed. O'Reilly Media, Inc, Sebastopol, CA.